BLOGGER CONTENT AUTOMATION ENGINE

Repair and improvement specification for Google Gemini / AI Studio

Purpose: Give Gemini clear, file-by-file instructions to repair the existing repository—not rebuild it from scratch—and make the pipeline safer, more reliable, and capable of producing useful articles in plain, natural English.

Important writing rule: The articles must sound like a clear, helpful human wrote them. Use ordinary words, short explanations, real examples, and a natural voice. Avoid bombastic words, filler, clichés, fake enthusiasm, and generic AI-style phrases.

Prepared from the repository review and test run described in the accompanying conversation. This is a repair brief and acceptance checklist; Gemini must inspect the current files before editing and must not assume a feature works just because a dry run or unit test passes.

1. Goal and rules for the repair

Work on the existing Python Blogger automation project. Keep its overall purpose and useful modules, but fix the issues listed below. Do not simply write a new demo, replace real integrations with mocks, or claim that something works without testing it.

- **Default publishing mode:** draft only. Never publish public posts unless the owner deliberately enables that mode.
- **Fail safely:** if research, citations, article generation, image handling, quality checks, or Blogger authentication fails, stop the run or save a clearly marked report. Do not quietly invent data or publish a fallback article.
- **Secrets:** use environment variables or GitHub Actions secrets. Never put API keys, refresh tokens, client secrets, passwords, or personal data in source files, logs, screenshots, or committed output.
- **Preserve useful work:** make focused changes to existing files and explain any new files. Avoid needless rewrites.
- **Evidence:** after changes, report the exact files changed, tests run, results, remaining limitations, and any manual setup still required.

2. The writing style is a core requirement

Every article, tutorial, intro, heading, example, summary, and call to action must follow these rules. Treat them as required quality checks, not optional style suggestions.

Write like a helpful person

- Use plain, simple English that a general reader can understand, including a reader who is new to AI, freelancing, or online business.
- Prefer familiar words. If a technical term is needed, explain it the first time it appears.
- Use direct sentences, mostly short or medium length. Vary sentence length so the writing does not sound mechanical.
- Explain one idea at a time. Use practical examples, step-by-step instructions, and realistic warnings about costs, limits, and risks.
- Be specific. Say what a tool does, who it helps, how to try it, and what its limits are. Do not make broad claims without evidence.
- Use a calm, friendly, useful tone. Do not force jokes, excitement, or fake personal stories.
- Use Zambian examples when genuinely useful—such as local payment access, data costs, mobile-first use, or opportunities available to people in Zambia—but do not pretend every method works locally without checking.

Avoid AI-sounding or generic wording

Avoid phrases such as these unless a rare, specific context truly requires them:

```
In today's fast-paced digital world...
In the ever-evolving landscape of...
Unlock the power of...
Revolutionize the way you...
Dive into / delve into...
Let's embark on a journey...
Game-changer / groundbreaking / cutting-edge
Seamlessly / robust / comprehensive (when a plain word works)
It's important to note that...
Whether you're a beginner or an expert...
Look no further...
In conclusion... (especially as a repeated, empty ending)
This ultimate guide will transform your life...
```

This list is not a blind banned-word filter. The real rule is to avoid empty, inflated, repetitive wording. Rewrite the idea in a natural way rather than swapping in another fancy phrase.

Example: before and after

Too generic / inflated	Preferred
“In today's fast-paced digital landscape, AI tools are revolutionizing the way freelancers unlock their full potential.”	“AI tools can help freelancers draft emails, plan projects, and finish some tasks faster. You still need to check the work before sending it to a client.”
“Embark on your journey to financial freedom with these game-changing strategies.”	“Here are a few ways to look for online work. None guarantees income, and some take time to learn.”
“This comprehensive guide will delve into the intricacies of freelancing.”	“This guide explains how freelancing works and how to get started.”

Human-sounding quality gate

- Detect repeated phrases, repeated sentence openings, excessive transition words, unsupported first-person claims, and generic filler.
- Do not judge naturalness using a single AI detector or claim that a detector can prove who wrote an article. Use a style checklist and targeted tests.
- Do not invent personal experiences, quotes, testimonials, client results, earnings, or hands-on testing. If the author has not tested a tool, say so or describe the information as research.
- Before accepting an article, ask: Would a real beginner understand this? Does every paragraph help? Could a simpler phrase say the same thing? Are claims supported? Remove anything that fails.
- Add unit tests for the style rules and a small set of example articles. Tests should flag obvious filler, not reject every normal use of a word.

3. Fix the live research and source verification

Files to inspect: `src/research.py`, `src/topic_discovery.py`, `src/article_writer.py`, `src/quality_checks.py`, and any configuration or prompt files.

- Remove keyword-based hardcoded research presented as if it were current research. Static examples may remain only as clearly labelled test fixtures.
- Implement real research using the configured provider or a clearly documented source workflow. Keep credentials in secrets.
- For every source, check that the URL is valid and reachable when practical, retrieve page content or useful metadata, record the page title and access date, and capture the specific evidence used.
- A failed HEAD request must not automatically mean a URL is valid. Try a sensible GET fallback; if verification still fails, mark the source unverified and do not count it as verified evidence.
- Verify that each factual claim is supported by the cited source. Do not cite a homepage for a specific claim if the supporting detail is not there.
- Do not make up URLs, citations, dates, statistics, product features, fees, or earnings claims. If a fact cannot be confirmed, remove it, qualify it, or explain that it could not be verified.
- Fix the stale Upwork service-fee statement. Do not claim a flat 10% fee across all contracts. Check the current official pricing/help page when writing about fees and state the date checked.
- Store source title, URL, retrieval time, relevant excerpt/evidence, and claim-to-source mapping in structured data so QA can inspect them.
- Add timeouts, retry limits, rate-limit handling, and clear error states. Do not loop indefinitely or silently replace failed research with made-up material.

4. Fix screenshots and image handling

Files to inspect: `src/screenshot_capture.py`, `src/main.py`, `src/seo_metadata.py`, and `src/blogger_client.py`.

- The current flow passes a local path such as `output/screenshots/example.png` into article HTML as if it were a public image URL. That path will not work for readers.
- Choose one real approach: upload the screenshot to a supported public HTTPS image host and use its returned URL, or omit the screenshot from published HTML. Do not insert a runner-local path.
- Check that the image is actually created, is a valid image, has a sensible file size and dimensions, and is not a tiny placeholder.
- If screenshots are required by the topic, failure to create or host them must fail the relevant quality gate. Do not let generated article metadata silently turn a required screenshot off.
- Only use screenshots that are allowed by the target website's terms and technical restrictions. Avoid capturing private data, login-only content, personal information, or pages that disallow automated access.
- If no reliable image host is configured, keep the pipeline draft-only and clearly report that the image is missing rather than embedding a broken image.

5. Replace the broken OAuth helper

File to inspect: `scripts/get_blogger_refresh_token.py`, plus Blogger authentication code and setup documentation.

- The helper currently uses Google's deprecated out-of-band redirect value `urn:ietf:wg:oauth:2.0:oob`, while defining a different redirect URI that is not actually used.
- Replace it with a supported OAuth flow. For a local setup script, use the appropriate installed-app/loopback flow supported by Google's current libraries and client type, or use a properly configured web redirect if the app is designed as a web app.
- Ensure the redirect URI used in the authorization request matches the configured OAuth client and the token exchange.
- Never ask the user to paste a client secret or refresh token into a public chat, commit it to the repository, or print it in logs.
- Document the Google Cloud steps, Blogger API enablement, OAuth consent/testing requirements if applicable, client type, redirect setup, and exact secret names expected by GitHub Actions.
- Add tests for missing credentials and authentication errors. Do not pretend an OAuth flow succeeded in dry-run mode.

6. Make topic history persist between GitHub Actions runs

Files to inspect: `src/state_store.py`, `src/topic_discovery.py`, `src/main.py`, and `.github/workflows/content-pipeline.yml`.

- The current JSON state is stored on the runner's local disk. GitHub-hosted runners are temporary, so the history can disappear after each run.
- Choose and document a durable approach. For a low-cost setup, one option is to restore/save a small state file as a GitHub Actions artifact or cache, but do not rely on a cache as the only correctness guarantee. Another option is a dedicated persistent store or a carefully controlled commit to a state branch.
- Make topic selection consult the persistent history before spending research or writing API quota. Do not repeatedly choose the same first topic after each scheduled run.
- Use stable topic IDs or normalized topic fingerprints, not only exact titles. Record status, timestamps, and any Blogger post ID.

- Protect concurrent runs with workflow concurrency settings or a lock so two runs cannot process the same topic at once.
- Add tests proving that a topic processed in one run is skipped in a later run when state is restored.

7. Make Blogger posting idempotent and safe

Files to inspect: `src/blogger_client.py`, `src/main.py`, `src/state_store.py`, and the workflow.

- Keep draft-only mode as the default and make live publishing require an explicit, clearly documented setting.
- Check for an existing post before generating expensive content where possible, and check again before insertion. Use a stable topic fingerprint and Blogger post ID when available.
- Do not swallow Blogger lookup errors and then continue as if no matching post exists. If reconciliation fails, stop safely rather than risk creating duplicates.
- Handle timeouts, HTTP errors, quota limits, expired credentials, and retryable errors with bounded retries and useful logs.
- Make retrying a failed run safe: a retry must not create another post for a topic that already has a draft or published post.
- Use the official Blogger API request/response shape. Confirm that the blog ID, title, content, labels, status, and post ID are handled correctly.
- Test with mocked API responses and, before any public publishing is enabled, perform a manual test that creates a draft in a test blog or a clearly identified test post.

8. Gemini response handling must fail closed

File to inspect: `src/article_writer.py` and related prompts/configuration.

- If Gemini returns malformed JSON, missing fields, empty content, or an unexpected response, do not replace it with a generic fallback article and continue.
- Validate the response against a strict schema. Retry a small, bounded number of times with a repair prompt if useful; if it still fails, mark the run failed.
- Keep dry-run fixtures clearly separate from live generation. A test article must never be mistaken for a real article in a live workflow.
- Record model/provider errors without logging secrets or unnecessarily storing private request data.
- Add tests for malformed JSON, missing title, empty body, missing citations, and API errors.

9. Strengthen article quality and SEO checks

Files to inspect: `src/quality_checks.py`, `src/seo_metadata.py`, `src/article_writer.py`, and tests.

- Check minimum useful content, a clear title, heading structure, working links, source quality, supported claims, readability, repeated content, and missing required sections.
- Do not treat a URL that starts with http/https as proof of a good source. Validate the actual source and evidence.
- Check that citations appear near the claims they support and that each cited source exists in the research record.
- Add a duplicate-content check using normalized text or a content fingerprint, not title matching alone.
- Use accurate SEO title, description, slug/permalink suggestions, labels, and image alt text. Do not keyword-stuff or create misleading titles.
- Validate HTML and ensure generated output does not contain internal local file paths, debug text, placeholder URLs, mock data, or secrets.
- A mock screenshot, fake source, or dry-run article must not pass live-mode quality checks.
- If affiliate links or paid recommendations are ever used, include a clear disclosure where required. Do not insert affiliate links automatically unless they are supplied and approved by the owner.

- Keep quality results explainable: list each failed check and what needs fixing. Do not report every stage as successful when it was skipped or mocked.

10. Secure the AI Studio developer console

Files to inspect: `server.ts`, `src/App.tsx`, `package.json`, and any routes exposed by the console.

- The current console includes a pipeline-run route without visible authentication. Do not expose this server publicly as-is.
- The server constructs a shell command using `execAsync(cmd)` and interpolates a supplied topic ID. Replace shell-string execution with a safe process-spawn API that passes arguments separately, and validate all user-controlled input.
- Protect pipeline execution and file-reading routes with authentication and authorization, or remove the console from the production automation bundle.
- Restrict file reads to an explicit allowlist of files within the repository. Block path traversal and prevent reading secrets, environment files, private keys, and unrelated files.
- Require deliberate confirmation for live mode. Never let an unauthenticated request start a live pipeline.
- Keep the clean Python automation repository separate from the developer UI bundle. The nested `public/blogger-content-engine.zip` was the cleaner Python-only package in the reviewed archive; use it as a starting point only after applying and testing the repairs.

11. GitHub Actions, dependencies, and reliability

Files to inspect: `.github/workflows/content-pipeline.yml`, `.github/workflows/tests.yml`, `requirements.txt`, and README/setup docs.

- Keep the scheduled workflow draft-only by default. Require minimal permissions; do not grant write access unless the chosen state-persistence method needs it.
- Add workflow concurrency controls, job timeouts, bounded retries, and clear step summaries.
- Pin GitHub Actions to full commit SHAs where practical, and document how those pins are updated.
- Review dependencies for actual use, compatible versions, and security. The current requirements comments say packages are pinned while version specifiers use >=; make the documentation accurate and choose a reproducible strategy.
- Use a project virtual environment for tests so unrelated packages in the machine-wide Python environment do not affect the result.
- Keep secrets in GitHub Actions Secrets and pass only the required variables to the workflow.
- Ensure logs and reports say whether each stage ran for real, was skipped, or used a test fixture.

12. Required tests and proof

The previously reviewed archive passed 14 tests and Python compile checks, but its dry run used hardcoded research, a tiny mock PNG, and a simulated Blogger draft. Those checks do not prove that live research, real screenshots, OAuth, Gemini, or Blogger posting work. Repeat the tests after fixing the code and add the following coverage:

- Unit tests for plain-English style checks and generic filler detection.
- Tests proving research sources are verified and claim-to-source links are present.
- Tests proving an unreachable source is not marked verified.
- Tests proving screenshot local paths never enter published HTML.
- Tests for missing screenshot host/configuration and invalid image files.
- Tests for supported OAuth configuration and missing/invalid credentials.
- Tests proving topic history survives separate workflow runs.

- Tests proving duplicate topics/posts are not inserted after retries or lookup failures.
- Tests proving malformed Gemini JSON fails safely and cannot create a fallback live post.
- Tests for Blogger API errors, quota limits, and bounded retries.
- Tests proving mock/dry-run output cannot pass live-mode gates.
- Security tests for command injection, path traversal, and unauthorized console access.
- A full dry run that clearly labels every mocked stage.
- A manual end-to-end draft test using real configured credentials, without public publishing.

13. Acceptance criteria — do not call the job finished until these pass

Area	Must be true
Writing	Articles use plain, clear English; inflated phrases and filler are reduced; no invented personal stories or earnings claims.
Research	Sources are real and checked; key claims are supported; failed verification is visible and blocks unsafe output.
Images	Every image in article HTML has a valid public HTTPS URL, or the image is omitted.
OAuth	The helper uses a supported flow and the documented redirect/client settings match.
State	Topic history survives scheduled runs and prevents repeat processing.
Blogger	Lookup failures stop safely; retries do not create duplicates; draft mode remains default.
Gemini	Malformed output fails safely; no generic fallback article is passed off as live content.
Quality	QA checks claims, sources, HTML, duplicates, placeholders, and writing style.
Security	No unauthenticated live run, shell injection, unsafe file reads, or secrets in the repo/logs.
Workflow	Least privilege, concurrency, timeouts, clear reports, and reliable dependency setup.
Proof	Tests, compile checks, and a documented manual draft test are reported honestly.

14. Copy-ready master prompt for Google Gemini / AI Studio

Paste the prompt below after uploading or opening the current repository. Ask Gemini to edit the actual files, not merely give advice.

```
You are a senior Python automation engineer, security reviewer, and plain-English editor. Repair the EXISTING Blogger Content Automation Engine repository that I provide. Do not rebuild it from scratch, do not replace real features with mocks, and do not claim a live integration works unless you have tested it.

PRIMARY GOAL Create a reliable GitHub Actions pipeline that discovers topics, researches them with real sources, writes useful articles with Gemini, optionally captures and hosts screenshots, runs strict quality checks, and creates Blogger DRAFTS through the real Blogger API. Draft-only must remain the default. Public publishing must require an explicit owner-controlled setting.

MANDATORY WRITING STYLE All article content must use plain, simple English that a general reader can understand. Write naturally, like a helpful human explaining something clearly. Prefer familiar words, short or medium sentences, practical examples, and direct instructions. Avoid bombastic language, hype, clichés, repetitive transitions, empty introductions, generic AI-sounding phrases, and filler such as “In today's fast-paced digital world,” “unlock the power,” “delve into,” “game-changer,” “seamlessly,” and “embark on a journey.” Do not just replace these phrases with other fancy words. Rewrite the thought in a natural way. Do not invent personal experiences, testimonials, quotes, hands-on testing, earnings, or results. Explain technical terms when needed. Use Zambia-specific examples only when relevant and factually checked.

REPAIR THESE AREAS 1. src/research.py and src/topic_discovery.py: replace hardcoded pseudo-research with actual source discovery/retrieval. Verify URLs and evidence. A failed HEAD request must not return “verified”; use a GET fallback where appropriate. Keep title, URL, access date, evidence excerpt, and claim-to-source mapping. Do not invent facts or citations. Correct stale claims such as a flat 10% Upwork fee; check current official sources. 2. src/article_writer.py: enforce a structured response schema. If Gemini returns invalid JSON, missing fields, empty content, or an API error, retry a bounded number of times if useful, then fail safely. Never substitute a generic fallback article during a live run. 3. src/screenshot_capture.py, src/main.py, src/seo_metadata.py: never insert a runner-local screenshot path into Blogger HTML. Upload screenshots to a configured public HTTPS host and use the returned URL, or omit them. Validate image type, size, dimensions, and actual content. A required screenshot must not be silently disabled. 4. scripts/get_blogger_refresh_token.py: replace the deprecated urn:ietf:wg:oauth:2.0:oob flow with a currently supported OAuth flow. Ensure authorization and token-exchange redirect URIs match the configured client. Never print or commit secrets. Update setup instructions. 5. src/state_store.py, src/topic_discovery.py, .github/workflows/content-pipeline.yml: make processed-topic history persist across GitHub Actions runs. Use stable topic fingerprints, record status and Blogger post ID, prevent concurrent duplicate runs, and test that history survives a later run. 6. src/blogger_client.py and src/main.py: make posting idempotent. Do not swallow reconciliation errors and continue to insert. If checking for an existing post fails, stop safely. Use bounded retries for transient errors. Keep draft-only as default. 7. src/quality_checks.py and src/seo_metadata.py: verify claim support, real sources, citations near claims, links, HTML, duplicates, headings, title/description, placeholders, local paths, mock data, and writing style. An http/https prefix alone is not source verification. Mock assets and dry-run fixtures must never pass live gates. 8. server.ts and src/App.tsx: secure or remove the developer console from production. Do not use shell-string execution with untrusted input; use safe process spawning and validated arguments. Protect pipeline and file routes with authentication/authorization. Prevent path traversal and secret-file access. Do not expose unauthenticated live runs. 9. .github/workflows/content-pipeline.yml and tests.yml: retain least privilege, add concurrency control, timeouts, bounded retries, clear run summaries, and safe secrets handling. Pin Actions to full SHAs where practical. Make dependency versions reproducible and correct misleading comments. 10. Documentation: explain setup from zero, required Google Cloud/Blogger API setup, supported OAuth flow, all environment variable names, GitHub secrets, image-host configuration, draft/live behavior, persistent state, local tests, workflow scheduling, recovery from errors, and remaining limitations.

WORK METHOD - First inspect the entire repository and report a short plan tied to exact file paths. - Then make the changes in the existing files. Add new files only where they are needed. - Keep dry-run fixtures separate from live integrations and label mocked stages clearly. - Never commit or expose credentials. - Add tests for each repaired behavior, including malformed Gemini JSON, failed source verification, screenshot local paths, persistent topic state, duplicate Blogger posts, API errors, unsafe console access, command injection, and path traversal. - Run the test suite, Python compile checks, and any available lint/type checks. Fix failures rather than hiding them. - Do not claim a real end-to-end test if credentials or external services were not available. If a live test cannot be run, state exactly what remains unverified. - Do not change draft-only default or enable public publishing automatically.

FINAL REPORT REQUIRED Give me: (1) a file-by-file change list, (2) the reason for each change, (3) commands run and exact test results, (4) what was genuinely tested live versus mocked, (5) any manual setup I must complete, (6) any known limitations, and (7) confirmation that articles are written in plain, natural English. Do not say “all fixed” unless every acceptance criterion is verified.
```

15. Official references Gemini should check

Use current official documentation when implementing external integrations. These links are starting points, not a substitute for checking current requirements during the repair:

- Google OAuth out-of-band migration: https://developers.google.com/identity/protocols/oauth2/resources/oob-migration
- Google OAuth for native/desktop apps: https://developers.google.com/identity/protocols/oauth2/native-app
- Blogger API — insert a post: https://developers.google.com/blogger/docs/3.0/reference/posts/insert
- Upwork freelancer service fee help page: https://support.upwork.com/hc/en-us/articles/211062538-Learn-about-the-Freelancer-Service-Fee
- Upwork freelancer pricing: https://www.upwork.com/pricing/freelancer

The repository review previously found 14 passing tests and successful Python compile checks, but the dry run used mock research, a tiny mock PNG, and simulated Blogger behavior. The repair is complete only when the new acceptance criteria are met and unverified live integrations are clearly disclosed.